Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Puebla

TE3001B. Fundamentación de robótica (Gpo 101)

Reto semanal 3. Manchester Robotics (Lazo cerrado)

Alumnos:

- Bruno Alejandro Domínguez Maldonado | A01738844
- Iván Ariel Rebollar León | A01737891
- Omar Kuri Vázquez | A01738129
- Erik García Cruz | A01732440

19 de abril de 2026

Resumen

Este reporte documenta la transición de una navegación en lazo abierto a un sistema de control en lazo cerrado para el robot diferencial Puzzlebot bajo ROS 2. A diferencia de la etapa anterior, se implementó un algoritmo de control PID (Proporcional-Integral-Derivativo) para la velocidad lineal y un control P (Proporcional) para la orientación, utilizando la odometría calculada a partir de los encoders de las ruedas como retroalimentación constante. El sistema fue validado mediante el seguimiento de una trayectoria cuadrada de 2 metros por lado, logrando corregir perturbaciones y errores acumulados que el sistema previo no podía gestionar.

Objetivos

Objetivo General

Implementar un sistema de control de lazo cerrado robusto en ROS 2 que permita al Puzzlebot seguir trayectorias precisas, compensando errores de posición y orientación en tiempo real.

Objetivos Específicos

- Desarrollar un nodo de control (ClosedLoopCtrl) que integre la lectura de encoders para el cálculo de odometría.
- Diseñar e implementar un controlador PID lineal para el desplazamiento y un P angular para la orientación hacia el objetivo (Goal-to-Goal).
- Configurar una trayectoria de cuatro puntos para formar un cuadrado de 2×2 metros, partiendo de una pose inicial [0,0,0].
- Aplicar estrategias de robustez, como la limitación de saturación de velocidad y el control de integral windup.

Introducción

La navegación autónoma avanzada requiere que el robot sea capaz de percibir su estado actual y compararlo con un estado deseado. En la robótica móvil, esto se logra mediante:

- Odometría: Es el método para estimar la posición (x, y) y orientación (θ) del robot integrando las velocidades de las ruedas obtenidas por los encoders a lo largo del tiempo.
- Control en Lazo Cerrado: A diferencia del lazo abierto, este sistema utiliza el error (la diferencia entre el objetivo y la posición actual) para ajustar dinámicamente los comandos de velocidad.

- Controlador PID: Es un mecanismo de control por realimentación que calcula el error mediante tres términos: el Proporcional (error actual), el Integral (acumulación de errores pasados) y el Derivativo (predicción de errores futuros).
- Robustez: Se define como la capacidad del controlador para mantener el desempeño ante perturbaciones externas, ruidos en los sensores (encoders) y no linealidades de los motores.

En el ecosistema de ROS 2, la comunicación fluye mediante el 'odometry message' (nav_msgs/msg/Odometry), que empaqueta tanto la pose como la velocidad. Un punto técnico crucial que configuramos fue el QoS (Quality of Service). En lugar de usar una configuración genérica, se ajustaron los perfiles (priorizando la latencia sobre la fiabilidad) para asegurar que el nodo de control recibiera siempre la lectura más reciente de los sensores, evitando desfases temporales en el cálculo del error en tiempo real.

Al verificar que existía retraso al obtener datos mediante la suscripción al tópico /odom para el nodo de odometría, optamos por calcular la odometría mediante la suscripción a los encoders. Esto debido a que reduce la latencia. El robot calcula su propia posición (x, y, θ) basándose en los pulsos de los motores en tiempo real. Si la Hackerboard está saturada, es más confiable leer los datos crudos de los sensores que esperar a que otro nodo procese la odometría. Es decir, empleamos odometría por diferencial.

Odometría de un Robot Móvil Diferencial

Un robot diferencial posee dos ruedas motrices independientes separadas por una distancia l (la trocha o wheelbase). La velocidad tangencial de cada rueda se obtiene multiplicando su velocidad angular (ω) por el radio r de la rueda. A partir de estas dos velocidades tangenciales, es posible calcular las velocidades cinemáticas del robot mediante las relaciones:

$$V = \frac{(v_r + v_l)}{2}$$

$$\omega = \frac{(v_r - v_l)}{l}$$

donde V es la velocidad lineal del centro del robot y ω es su velocidad angular. Cuando ambas ruedas giran a la misma velocidad, el robot avanza en línea recta (ω = 0); cuando una rueda gira más rápido que la otra, el robot gira hacia el lado de la rueda más lenta. Estas relaciones constituyen la base del modelo cinemático diferencial (Quigley et al., 2015).}

Cálculo del Error en un Robot Móvil Diferencial

Una vez que el robot conoce su posición estimada (x, y, θ) gracias a la odometría, es posible calcular el error de posición respecto a un punto objetivo (x_d, y_d). El error de distancia, también llamado error lineal, representa qué tan lejos está el robot de su meta:

$$e_d = \sqrt{(x_d - x)^2 + (y_d - y)^2}$$

El error angular, por su parte, indica cuánto debe girar el robot para apuntar hacia el objetivo. Se calcula como la diferencia entre el ángulo deseado θ_d (obtenido mediante atan2) y la orientación actual θ del robot, normalizado al intervalo [−π, π] para evitar giros innecesarios. Estos dos errores constituyen las señales de entrada para cualquier controlador de lazo cerrado, como un controlador proporcional P o un PID (Siegwart et al., 2011).

QoS: Quality of Service en ROS2

La Calidad de Servicio (QoS, por sus siglas en inglés) es un mecanismo de ROS2 que permite configurar cómo se transmiten los mensajes entre publicadores y suscriptores. A diferencia de ROS 1, donde la comunicación se basaba en TCP con una semántica fija, ROS2 utiliza el middleware DDS (Data Distribution Service), que ofrece múltiples políticas configurables para adaptarse a distintos casos de uso (Open Robotics, 2024).

Las políticas de QoS más relevantes para aplicaciones de robótica móvil son la confiabilidad (Reliability), que puede ser RELIABLE —garantiza la entrega de todos los mensajes— o BEST_EFFORT —prioriza la velocidad sobre la garantía de entrega—; la durabilidad (Durability), que define si los mensajes se almacenan en caché para nuevos suscriptores; y la profundidad de la cola (History Depth), que determina cuántos mensajes recientes se conservan en memoria. Para datos de sensor de alta frecuencia, como los encoders del Puzzlebot, se emplea el perfil qos_profile_sensor_data de ROS2, que combina BEST_EFFORT con una profundidad de cola de 10. Esto minimiza la latencia a costa de tolerar pérdidas ocasionales de paquetes, lo cual es aceptable cuando los datos se renuevan continuamente a alta frecuencia (Lalancette et al., 2024).

El Método de Euler para Integración de Posición

El método de Euler es una técnica de integración numérica de primer orden que permite aproximar la solución de una ecuación diferencial a partir de su valor y su derivada en un instante dado. Para un robot diferencial, las ecuaciones de movimiento en el plano son:

$$\dot{x} = V \cdot \cos(\theta)$$

$$\dot{y} = V \cdot \sin(\theta)$$

$$\dot{\theta} = \omega$$

Aplicando el método de Euler con un paso de tiempo Δt, la posición en el instante siguiente se calcula como $x(t + \Delta t) = x(t) + \dot{x}(t) \cdot \Delta t$. Su principal ventaja es la eficiencia computacional y la sencillez de implementación, lo que lo hace idóneo para computadoras embebidas como la NVIDIA Jetson Nano. Aunque introduce un error de truncamiento proporcional a Δt, este se minimiza usando períodos de muestreo pequeños.

Definición de Robustez (Control de Robots Móviles)

En nuestro contexto del control controlador de lazo cerrado para un robot diferencial, la robustez es la capacidad del controlador para cumplir con el objetivo (llegar a los waypoints) manteniendo la estabilidad y precisión

Para la parte 1 del reto

En la primera parte del reto (cuadrado de 2X2 metros) la robustez se manifiesta específicamente en tres áreas:

Rechazo a Perturbaciones: es la capacidad de corregir la trayectoria si el robot patina o si el suelo tiene irregularidades. Al usar lazo cerrado, si el robot se desvía de la línea recta, el error de ángulo (e_{ang}) aumenta y el controlador reacciona inmediatamente ajustando cmd.angular.z para reorientarse.

Mitigación de No-Linealidades (Anti-Windup): los motores reales tienen límites. En el código, la robustez se implementa mediante el i_windup_limit y el np.clip. Sin esto, el término Integral del PID podría crecer infinitamente si el robot se bloquea (por ejemplo, al chocar con un obstáculo), lo que causaría una respuesta violenta o inestable al liberarse.

Inmunidad al Ruido de Sensores: los encoders suelen tener "ruido" o lecturas saltarinas. Un sistema robusto utiliza ganancias sintonizadas (como kd_v) que filtran o suavizan estas variaciones para evitar que los motores vibren o se desgasten prematuramente.

Solución del problema

Metodología

Para resolver este reto, el enfoque fue pasar de simples lecturas de sensores a una estimación de estado completa que permitiera saber dónde está el Puzzlebot en todo momento.

Fase 1: Implementación del Controlador PID

Parte 1: cuadro de 2X2 metros

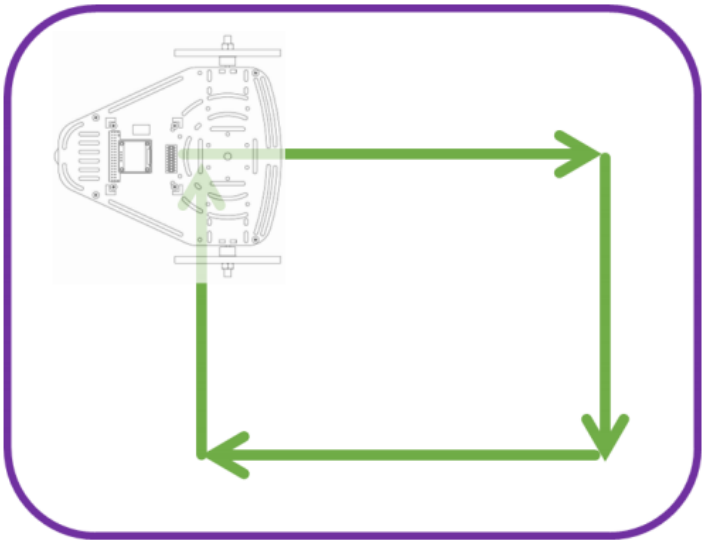

Figura 1: trayectoria que debe seguir el Puzzlebot en la parte 1 del reto. Cuadro de 2X2 metros.

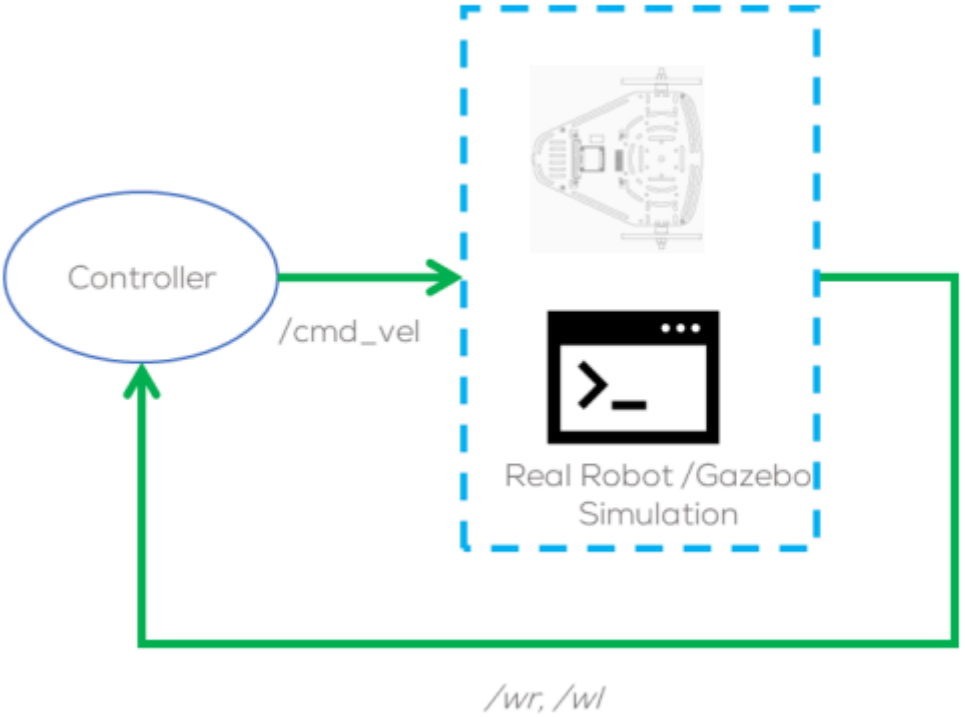

Figura 2: diagrama del nodo controlador y de los tópicos que se conectan. Los tópicos /wr y /wl son para el caso del simulador Gazebo, pero no fue empleado éste.

Se desarrolló el nodo ClosedLoopCtrl.py que realiza las siguientes tareas:

- Cálculo de Pose: Mediante la suscripción a /VelocityEncR y /VelocityEncL, se calcula la velocidad lineal v y angular ω. Se utiliza el modelo cinemático diferencial para actualizar (x, y, θ) en cada paso de tiempo dt.
- Lógica Goal-to-Goal: * Control Angular (P): Se calcula el ángulo hacia la meta. Si el error es mayor a 0.45 rad, el robot prioriza la rotación sobre el avance.

- Control Lineal (PID): Una vez orientado, se aplica el PID. Se implementó un Anti-windup limitando la suma del error integral (i_windup_limit) para evitar saturaciones descontroladas.

Parte 2: Generación de Trayectorias y Mensajes Personalizados (General)

Primero el nodo generador de trayectorias (path_generator) que es un nodo independiente lee los puntos de interés desde un archivo de configuración y los publica secuencialmente. Existe un mensaje personalizado que se define como un tipo de mensaje basado en geometry_msgs/Pose para comunicar las metas al controlador a través de un tópico llamado /goals. Aunado a esto cuenta con desacoplamiento, lo que permite que el controlador sea agnóstico a la ruta, reaccionando únicamente a la meta actual publicada en el bus de ROS 2.

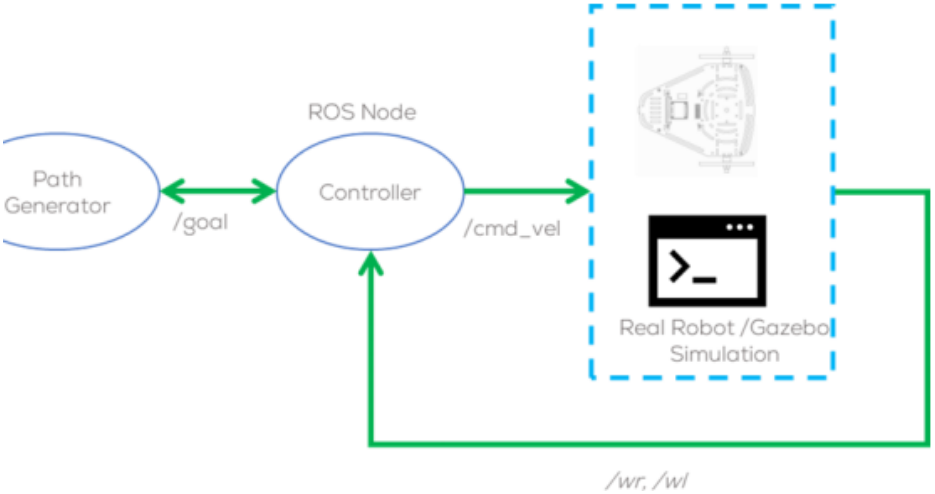

Figura 3: diagrama de nodos y tópicos a emplear en la segunda parte del reto. Los tópicos /wr y /wl son para el caso de emplear el simulador Gazebo, el cual no fue utilizado.

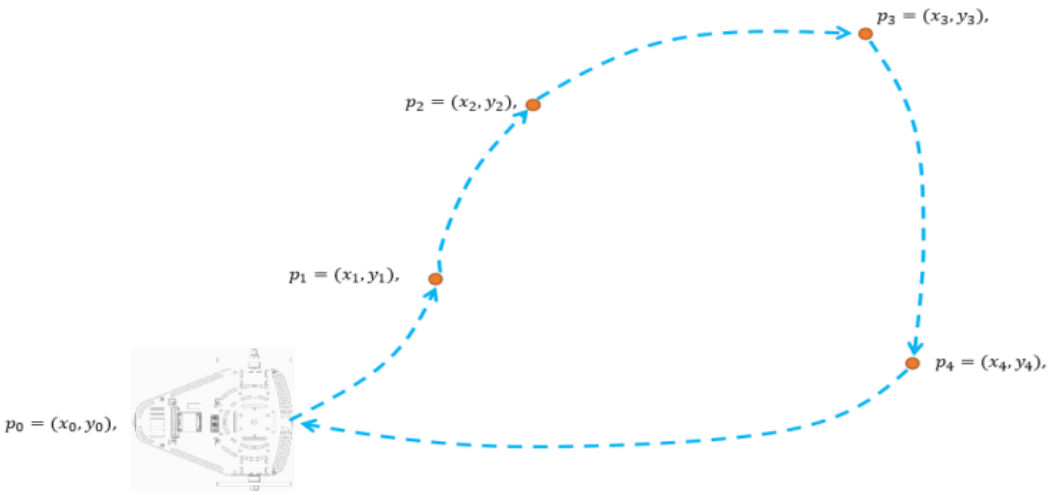

Figura 4: representación de una posible figura a emplear para la parte 2 del reto.

Fase 2: Integración de Posición con el Método de Euler

La segunda fase extiende el nodo base para integrar las velocidades calculadas y estimar la pose del robot en el plano. En la versión base provista, el nodo ya calcula V y ω, pero no actualiza las variables de posición X, Y, Th. La ampliación consiste en añadir dentro de la condición dt > self._sample_time el bloque de integración de Euler:

- self.X += self.V * np.cos(self.Th) * dt
- self.Y += self.V * np.sin(self.Th) * dt
- self.Th += self.Omega * dt

Esta implementación actualiza las coordenadas del robot en cada iteración, utilizando como entrada las velocidades estimadas en ese mismo instante y el intervalo de tiempo real dt obtenido del reloj del sistema. El uso del reloj real (get_clock().now()) en lugar de un dt fijo es importante porque ROS2 no garantiza que el timer se dispare con exactitud milimétrica; usar el tiempo real medido mejora la precisión de la integración (Siegwart et al., 2011).

Fase 3: Cálculo y Visualización del Error (Actividad 3.1)

La tercera fase añade al sistema la capacidad de cuantificar el error de posición respecto a un punto objetivo, sentando la base para futuros controladores de lazo cerrado. Dado un objetivo (x_d, y_d), el nodo calcula en cada iteración el error de distancia e_d y el error angular e_θ mediante:

$$e_d = \sqrt{(x_d - X)^2 + (y_d - Y)^2}$$

$$e_\theta = atan2(y_d - Y,\ x_d - X) - \theta$$

El error angular resultante se normaliza al intervalo [−π, π] mediante la función wrap_to_pi, análoga a la empleada en el reto anterior, para garantizar que el robot siempre elija el giro más corto.

Resultados

Parte 1

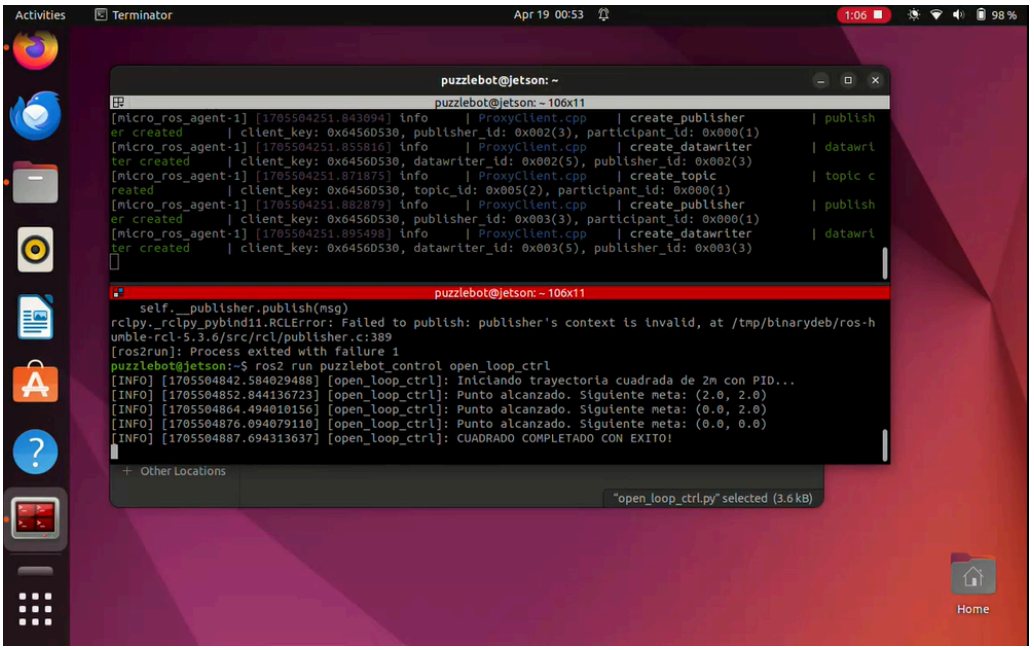

Figura 5. Puntos alcanzados en la terminal para el caso del cuadro de 2X2 m.

Vídeo de la terminal durante el recorrido del puzzlebot: https://drive.google.com/file/d/1tYJ4sh5T3P9jYJ71EHq3XdNHLZVvKhFS/view

En la terminal se muestra cómo el Puzzlebot va alcanzando los puntos del cuadro de 2X2 metros.

Vídeo del Puzzlebot realizando el recorrido: https://drive.google.com/file/d/1k4SGbcxYfak3-kVkCzbR_NYOTIB9jKlv/view?usp=sharing

En el vídeo se observa como el Puzzlebot realiza la trayectoria del cuadro. Avanza 2 metros, gira 90° y luego avanza de nuevo hasta terminar dicha trayectoria.

Los resultados muestran que el sistema de lazo cerrado cumplió su función correctamente. Como se aprecia en la terminal, el Puzzlebot fue alcanzando cada uno de los cuatro waypoints del cuadrado de forma secuencial, lo que confirma que la odometría calculada desde los encoders fue suficientemente precisa para guiar al robot en tiempo real. El controlador PID corregía continuamente la velocidad lineal mientras que el P angular re-orientaba al robot antes de cada tramo, resultando en una trayectoria visualmente bastante cercana al cuadrado ideal de 2×2 metros.

Parte 2

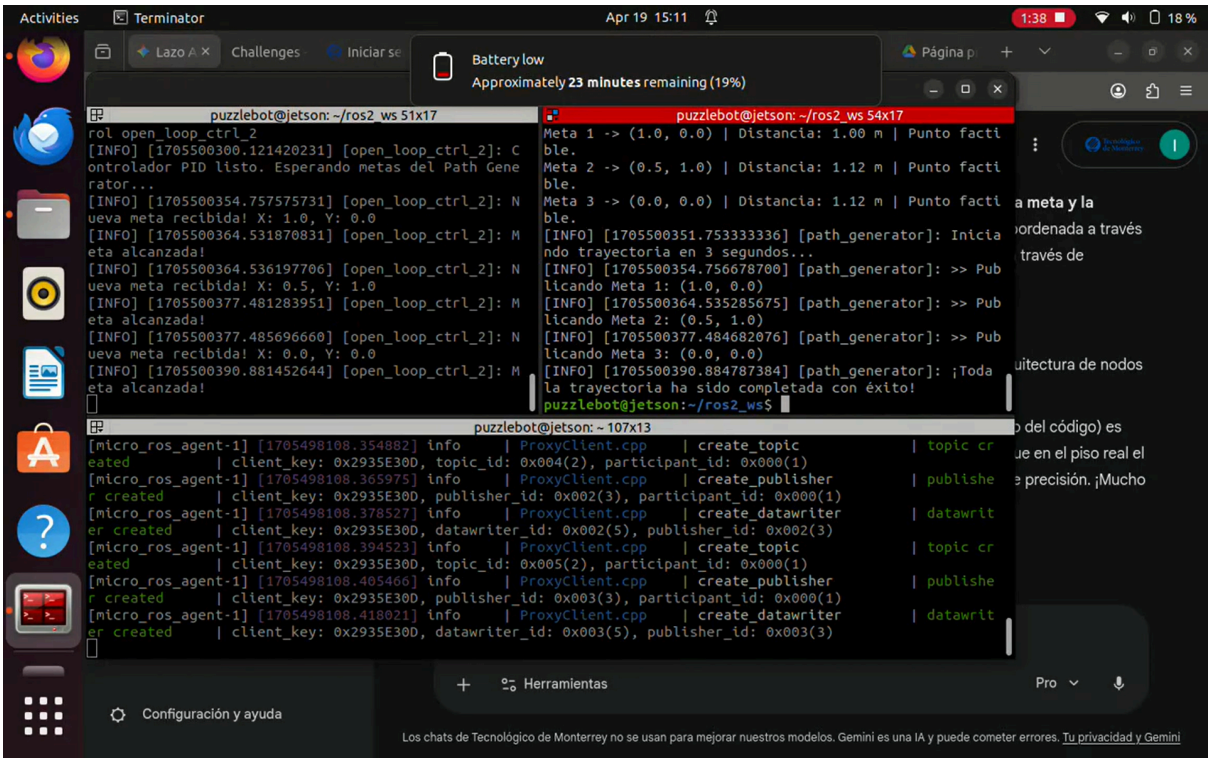

Figura 6. Terminales de la Jetson Nano durante la ejecución de la Parte 2, evidenciando la comunicación entre el generador de trayectorias y el controlador de lazo cerrado.

Vídeo de las terminales en la laptop: https://drive.google.com/file/d/1SFdzcHxa9Qqhh-in0jYxckp-vdR9o8HC/view?usp=sharing

En la terminal se aprecia cómo se van ingresando los puntos que el robot debe seguir y cómo posteriormente el Puzzlebot cumple esas trayectorias.

Vídeo con el movimiento del Puzzlebot siguiendo los puntos marcados: https://drive.google.com/file/d/1gjqn5HUzt4NQcl-OpphF5KOPSnuZNucS/view?usp=sharing

En el video se observa cómo el Puzzlebot ejecuta la trayectoria a partir de múltiples metas generadas. El robot se desplaza hacia cada punto objetivo, ajustando su dirección en cada cambio, hasta completar toda la trayectoria de manera secuencial.

La arquitectura de dos nodos demostró funcionar correctamente para trayectorias con múltiples metas. El nodo path_generator publicó los waypoints secuencialmente y el controlador los procesó uno a uno, ajustando su orientación antes de cada desplazamiento. El desacoplamiento entre planificación y ejecución resultó especialmente útil durante las pruebas, ya que fue posible modificar la trayectoria sin tocar la lógica del controlador. Sin

embargo, en recorridos con más cambios de dirección los errores de orientación entre waypoints fueron más notorios que en la parte 1, lo que indica que este tipo de trayectorias más complejas representan el caso de uso más exigente para el sistema implementado.

Conclusiones

En el reto anterior trabajamos con navegación en lazo abierto, donde el Puzzlebot ejecutaba comandos de tiempo y velocidad precalculados sin conocer su posición real. Cualquier perturbación, como un deslizamiento o irregularidad en el piso, se acumulaba sin posibilidad de corrección. Con este reto, ese problema se resolvió: al implementar el controlador PID con odometría calculada directamente desde los encoders, el robot ahora percibe su estado real y ajusta su trayectoria en tiempo real. En ese sentido, el objetivo general se cumplió satisfactoriamente, ya que el Puzzlebot logró seguir la trayectoria cuadrada de 2×2 metros corrigiendo desviaciones que en el reto anterior habrían arruinado el recorrido completo.

Respecto a los objetivos específicos, la mayoría se alcanzaron. El nodo ClosedLoopCtrl quedó funcional con odometría diferencial, el controlador PID lineal y el P angular respondieron correctamente bajo la lógica Goal-to-Goal, y las estrategias de anti-windup y saturación de velocidad demostraron ser indispensables durante las pruebas físicas. Sin embargo, hubo un objetivo que presentó complicaciones: inicialmente intentamos obtener la odometría desde el tópico /odom, pero el retraso que introducía afectaba directamente la respuesta del controlador. Esto nos obligó a recalcular la odometría desde los encoders crudos, lo cual funcionó mejor pero implicó tiempo extra de depuración que no teníamos contemplado.

Como posible mejora, consideramos que la precisión del sistema podría aumentarse implementando un método de integración de orden superior, como Runge-Kutta, en lugar del método de Euler que usamos actualmente. Euler es funcional y computacionalmente ligero, pero introduce un error de truncamiento que se acumula en trayectorias largas. Adicionalmente, una sintonización más sistemática de las ganancias PID, mediante algún método formal como Ziegler-Nichols en lugar del ajuste manual que realizamos, probablemente reduciría el tiempo de asentamiento y mejoraría la precisión al llegar a cada waypoint.

Referencias

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357-362. https://doi.org/10.1038/s41586-020-2649-2

Lalancette, C., Hendrix, A., & Jay, G. T. (2024). teleop_twist_keyboard. ROS 2 Documentation. Recuperado de https://docs.ros.org/en/ros2_packages/humble/api/teleop_twist_keyboard/index.html

micro-ROS. (s.f.). ROS 2 feature comparison. micro-ROS Documentation. Recuperado de https://micro.ros.org/docs/overview/ROS_2_feature_comparison/

Open Robotics. (2024). ROS 2 documentation: Humble Hawksbill. ROS 2 Project. Recuperado de https://docs.ros.org/en/humble/index.html

Quigley, M., Gerkey, B., & Smart, W. D. (2015). Programming Robots with ROS: a practical introduction to the Robot Operating System. O'Reilly Media, Inc.

Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). Introduction to autonomous mobile robots. MIT press.